

条款

全AES的缓存未命中与恢复 256 钥匙

Samira Briongos^{1,2,*} , 佩德罗·马拉贡^{1,2,*} 胡安-马里亚诺·德·戈耶内切^{1,2} 和
何塞·M·莫亚^{1,2} 

¹ 西班牙马德里理工大学综合系统实验室, ETSI 电信, 28040 马德里; goyeneche@die.upm.es (J.-M.d.G.); josem@die.upm.es (J.M.M.)

² 马德里理工大学计算模拟中心, 蒙特加斯多校区, 西班牙马德里28660

* 通讯作者: samirabrigos@die.upm.es (S.B.); malagon@die.upm.es (P.M.)

接收日期: 2019年2月8日; 接受日期: 2019年2月27日; 发表日期: 2019年3月6日



重点应用: 本研究提出了一种创新方法, 可有效利用缓存攻击获取的信息。与传统方法不同, 我们通过分析缓存未命中现象来更高效地提取AES加密密钥: 针对OpenSSL AES r-表实现方案, 我们证明该方法能大幅减少样本量, 从而显著缩短获取128位密钥所需的时间。此外, 我们还成功实现了256位AES密钥的实际恢复。

摘要: CPU缓存是一种会泄露运行在CPU上的软件重要信息的硬件元件。特别是, 任何执行依赖敏感信息(如私钥)的内存访问序列的应用程序, 都容易遭受缓存攻击, 从而暴露这些信息。在大多数情况下, 侧信道缓存攻击不需要任何特定权限, 只需访问共享缓存即可。这一事实, 加上云计算基础设施在不同客户之间共享的普及, 使得此类攻击变得相当普遍。传统上, 针对AES的缓存攻击利用受害者信息来访问地址。相比之下, 我们证明了使用非访问方式能获取更多信息, 并指出近年来缓存攻击的威力被低估了。这种新方法适用于现有攻击: *Prime+Probe*、*Flush+Reload*、*Flush+Flush*和*Prime+Abort*。在所有案例中, 通过利用缓存未命中作为信息来源, 我们能够以比传统方法减少93%至98%的样本量, 成功获取128位AES密钥。此外, 我们对这种攻击方式进行了改进和扩展, 将其称为加密-解密缓存攻击(EBD), 从而获得256位AES密钥。在最佳情况下, 我们的方法仅需不到10,000次样本采集(即135毫秒)即可获取基于OpenSSL AES r表实现的256位密钥, 这证明通过缓存攻击攻击AES-256的复杂度仅比AES-128高出约三倍。此外, 该方法已在跨虚拟机场景中成功验证。

关键词: 侧信道缓存攻击; 缓存未命中; AES; 云计算

1. 引言

云计算的核心目标是根据用户实际需求提供计算资源, 从而省去购置和维护昂贵计算基础设施的麻烦。凭借其低成本、易用性以及按需获取计算资源的优势, 政府和企业对云计算技术的采用率持续攀升。亚马逊、谷歌和微软等科技巨头也纷纷加入这一行列。

转型为云服务提供商。云服务商凭借规模经济优势以低价提供计算能力：通过高效利用服务器资源，他们能将成本分摊给更多客户。这意味着多个虚拟机（VM）共享同一台物理主机，由虚拟机管理程序（VMM）实现逻辑隔离。

然而，租户之间并不存在物理隔离，因此共享硬件可能在不同虚拟机之间创建隐蔽通道。CPU缓存是这些共享资源之一，可被利用从共存租户中恢复细粒度信息。众所周知的攻击技术，如*Evict+Time*、*Prime+Probe*[1]、*Flush+Reload*[2]以及*Flush+Flush*[3]，或新近提出的*Prime+Abort*[4]，均可推断出受害程序访问的指令或数据。因此，当此类内存访问依赖敏感信息时，该信息便会泄露。

此类攻击从受害者处提取私有信息的能力，其最令人印象深刻的演示针对对称[1, 5, 6]与非对称[7–9]加密实现。由于其包容性架构和替换策略，英特尔CPU缓存传统上一直是缓存攻击的受害者。然而，多位研究人员[10–12]也证明，采用不同架构和替换策略的AMD与ARM处理器同样易受缓存攻击。

缓存攻击需要攻击者与受害者共享缓存。在云环境中，这意味着攻击者必须在攻击前与受害者实现共存。如Ristenpart等人首次所示[13]在知名云平台（如亚马逊EC2）中，共存现象既可实现又可检测。该研究成果发表后，云服务商对其采用的技术进行了混淆处理。然而最新研究[9, 14]表明，仍可判断两台虚拟机是否被分配在同一物理机上。当虚拟机与目标受害者被分配在同一主机时，其所有者便能获取邻近节点的活动信息。

研究人员也日益担忧这类攻击可能构成严重威胁。为此，业界提出了多种方案，试图避免数据泄露[15–17]或检测攻击[18–21]。据我们所知，目前在真实云环境中尚未实施任何硬件防护措施，也没有硬件制造商对缓存架构进行过调整。我们认为这种防护措施缺失的主要原因在于其带来的性能损耗。因此，想要利用侧信道缓存攻击的攻击者，只需考虑用户可实施的基于检测的防护措施即可。

本研究采用基于r表的AES实现方案。尽管该方案存在缓存攻击风险，但因其广泛使用，常被用作对比基准及攻击技术演示。通过该方案，我们旨在验证非内存访问信息获取的准确性，并量化其相较于传统基于内存访问方法的性能提升。

我们能够显著提升先前发表的侧信道攻击研究成果，将获取完整密钥所需的样本量降低多个数量级。所有已知技术（*刷新+重载*、*刷新+刷新主+探测*及*主+中止*）均可从该方法中获益。此外，本方法更不易产生误判的内存访问。对于非访问场景，若数据在短时间内被加载至缓存又迅速移除，则视为误判，这种情况极不可能发生。而传统方法的误判则出现在两种情形：一是数据在AES加密的其他轮次（而非目标轮次）被加载至缓存，二是数据被推测将被使用而被推测性加载至缓存——这两种情况都较为常见。

综上所述，本研究的主要贡献如下

- 我们提出了一种非访问攻击，这是一种利用缓存未命中信息获取新方法。
- 我们证明了我们的方法提高了先前缓存攻击的性能，并证明了其对*Flush+Reload*、*Flush+Flush*、*Prime+Probe*和*Prime+Abort*的有效性。我们可以减少

在93%至98%的加密量下即可推导出密钥。换言之，若访问攻击需要100,000次采样，我们的方法仅需不足3000次采样，且在类似真实实验环境中完成。

- 我们扩展了非访问攻击，通过引入 EBD 这一实用攻击实现，利用缓存侧信道攻击获取算法更多轮次的信息，从而获得完整的256位AES加密密钥。
- 研究表明，该攻击的复杂度不取决于密钥长度，而是取决于每次加密的轮数。从10轮（AES-128）增加到14轮（AES-256）会使复杂度增加约3倍。

本文其余部分的组织结构如下：第2节提供理解攻击的相关概念信息；第3节描述AES的实现方式及我们用于提取密钥的攻击方法；第4节对研究结果进行讨论；最后在第5节得出若干结论。

2. 背景及相关工作

为深入理解侧信道缓存攻击，我们首先简要介绍相关架构概念，继而详细阐述现有缓存攻击的原理与实施方式。

2.1. 建筑概念

本文阐述了三种关键缓存攻击特征的基本原理：CPU缓存、共享内存和事务内存。

2.1.1. 中央处理器缓存

CPU缓存是位于CPU与主内存之间的高速小型存储器，专门用于隐藏主内存访问延迟。由于内存成本与速度相关，缓存通常采用三级结构。一级缓存是最小且最快的存储器，与二级缓存类似，通常为各处理核心独享。三级缓存作为最大且最慢的存储器，通常由多个处理核心共享。它保存着近期使用过的数据副本，这些数据很可能在短时间内被处理器调用。若处理器请求的数据不在任何层级的缓存中，系统会从主内存加载数据并存储至缓存。当某级缓存完全填满时，部分现有数据将被替换（驱逐）以存储新加载的数据。驱逐数据的选择取决于处理器架构。包括AMD、英特尔及部分ARM高性能处理器在内的处理器，通常采用最近最少使用（LRU）策略。而大多数ARM处理器的替换策略则基于随机替换或轮询替换机制。

在英特尔处理器中，例如我们实验所用的型号，缓存属于包含式存储结构。这意味着三级缓存（L3）等高级缓存会保留低级缓存的数据副本。更重要的是，三级缓存是所有核心共享的资源，这使得某个核心正在执行的、涉及三级缓存数据的进程，可能会对其他核心的进程产生副作用。相比之下，其他处理器采用独占式缓存架构，即每个内存块仅存在于缓存层级结构的单一层级中。

缓存通常采用组相联结构：它们被组织为 S 组的 W 行（也称为通道），每行存储 B 字节。通常将其命名为 W 路组相联缓存。给定一个内存地址时，较低有效 $\log_2 B$ 位用于定位该行上的字节，前 $\log_2 S$ 位用于定位组中的字节，剩余高位位则作为每行的标签。该标签用于判断某行是否已被加载到缓存中。在现代处理器中，最后一级缓存（LLC）被划分为切片。因此，需要知道数据将被放置在哪个切片和组中。切片通过地址剩余位的哈希函数计算得出。如果核心数是2的幂，则哈希值将取决于标签位。否则，它将使用地址所有位的

地址[22 , 23]需注意, 由于主存容量远大于CPU缓存, 多条内存线会映射到同一缓存组。

2.1.2. 共享存储器

程序从硬盘加载到内存中执行时, 会同时使用内存存储指令和数据, 因为访问内存的速度比访问硬盘快得多。内存是有限的资源, 当没有空闲内存时, 部分数据和指令会被转移到虚拟内存——这是硬盘上存储的内存副本。在应用程序中过度使用虚拟内存会严重影响性能。操作系统采用内存共享等机制来降低内存占用是自然而然的。当两个不同进程使用同一库文件时, 内存共享机制可以将相同的物理内存页映射到每个进程的地址空间, 从而避免重复加载。

去重技术是一种具体的共享内存方法, 最初为提升虚拟机监控程序(VMM)的内存利用率而设计, 后来也被应用于非虚拟化环境。虚拟机监控程序或操作系统会扫描物理内存, 识别出将相同数据(即内容完全相同的页面)存入内存的进程。当多个页面恰好包含相同内容时, 系统会将所有指向这些相同页面的映射重定向到其中一个页面, 并释放其他页面。然而, 如果合并页面中的任一进程执行了任何修改操作, 内存就会再次被重复分配。

Linux内存去重功能的实现称为KSM(内核同页合并), 首次出现在Linux内核2.6.32版本中。KSM作为页面共享技术被内核虚拟机(KVM)采用, 我们在实验中将其用作虚拟机监控程序。KSM通过扫描用户内存来查找潜在的共享页面, 仅扫描潜在候选页面而非连续扫描整个内存[24]。

数据去重优化能节省内存资源, 让主机可以运行更多虚拟机。举个具体例子, 我们参考了onl[25]的研究成果, 他们指出在仅配备16GB内存的服务器上, 完全能够同时运行50多台Windows XP虚拟机, 每台都配备1GB内存。从性能角度来看, 数据去重确实是云服务商的卖点。但经过多次利用页面共享漏洞的侧信道攻击实证后, 业界普遍建议云服务商禁用该功能, 目前所有云服务商都已采纳这一建议。

2.1.3. 事务存储器

当多个线程并行运行并尝试访问共享资源时, 需要同步机制来避免冲突。事务内存技术旨在简化并发编程, 避免互斥带来的常见问题, 同时提升其他锁定技术的性能。该技术允许程序员指定的事务代码区域以乐观执行模式运行。处理器在执行指定代码段时, 会默认这些操作可以不受干扰地完成。程序员无需再负责识别锁的获取顺序, 只需明确哪些区域将被定义为“事务”的一部分即可。

在事务执行过程中, 变量和操作结果仅对当前线程可见, 这意味着这些区域内的任何更新操作都不会被其他线程察觉。若事务成功执行, 处理器会像即时生效般提交所有更改, 使这些变更对所有进程可见。反之, 若与其他进程发生冲突, 事务将失败并终止执行。因此, 处理器需要一种机制来撤销所有更新、放弃所有更改, 并将系统状态恢复到仿佛从未执行过该事务的初始状态。

英特尔事务同步扩展(TSX)是英特尔公司对硬件事务内存的实现方案。TSX提供两种软件接口: 硬件锁消除(HLE)和受限事务内存(RTM)。

2.2. 缓存攻击

缓存攻击会生成隐蔽通道，攻击者可利用这些通道窃取敏感信息。当进程尝试访问数据时，若数据已存在于缓存中（即缓存命中），其恢复时间将显著短于从主内存获取数据所需的时间（即缓存未命中）。因此，若加密进程的执行时间取决于数据在缓存中的存在状态，攻击者便可利用这种时间差从加密过程中获取私密信息（如密钥）。传统上，缓存攻击主要分为以下类型：

- **时间驱动攻击：**通过分析目标密码算法多次执行的时序特征来获取信息。
- **跟踪驱动攻击：**通过直接监控缓存获取信息，前提是攻击者在运行目标进程时能够访问缓存配置文件。
- **访问驱动型攻击：**该信息是从目标进程执行期间访问的缓存集合中获取的

缓存内存最早于1992年由Hu[26]被提及作为提取敏感信息的隐蔽通道。1996年，Kocher[27]提出了首个理论攻击，Kelsey[28]于1998年描述了基于缓存命中率实施攻击的可能性。2002年，Page[29]研究了DES缓存攻击的理论示例，该示例也被Tsunoo[30]用于研究因表查找产生的时序侧信道。2004年，Bernstein[5]在观察到执行加密算法时存在非恒定时间后，提出了首个时间驱动攻击。尽管他提出的攻击不具备实用性，但其相关性攻击已被广泛研究，甚至近期在虚拟机间重新尝试[31]。随后，Percival[32]提出另一种攻击，建议攻击者可通过测量缓存集所有通道的访问时间来确定其他进程占用的缓存通道。他发现这些时间与占用通道数量存在相关性。2006年，Osvik等人[1]提出了两种技术，自那时起已被广泛使用，分别命名为*Evict+Time*和*Prime+Probe*。这些技术旨在让攻击者能够确定受害者进程访问的缓存集。Gullasch[33]提出了一种利用共享内存和完全公平调度器（CFS）的更强大攻击。Yarom和Falkner[2]后来利用了相同的攻击原理，并将该攻击命名为*Flush+Reload*。该攻击的目标是L3缓存，因为它具有包容性且在核心之间共享。其中，*Flush+Reload*和*Prime+Probe*攻击（及其变体）因其更高的分辨率而脱颖而出。

2.2.1. 驱逐+时间

该技术包含三个步骤：首先触发加密并测量其执行时间；其次攻击者会清除部分缓存行；最后再次测量加密时间。通过对比两次测量结果，攻击者可判断该缓存行是否被访问——时间越长，说明该行被频繁使用。

2.2.2. 刷新+重新加载

该技术依赖于共享内存的存在：因此，当攻击者将目标行从缓存中刷新时，可以确保如果受害进程需要检索被刷新的行，就必须从主内存中加载。引发刷新+重装 还分为三个阶段：首先，从缓存中刷新目标行（可能需要逆向工程来确定可能泄露信息的地址）。其次，受害者运行其进程或其片段。最后，攻击者访问被刷新的行，并测量所需时间。根据重装时间，攻击者判断该行是否被访问。

该攻击利用*clflush*指令在初始化阶段清除目标行缓存。该技术实现简便且对微架构噪声具有较强抗扰性，因而广受青睐。但其主要缺陷在于需要进行内存去重操作。

重复数据消除功能已启用。该技术通过合并重复内存页来优化内存使用效率，但只能恢复静态分配数据中的信息。共享内存机制还要求攻击者与受害者必须使用相同的加密库。为避免攻击者需要掌握算法版本信息，格鲁斯等人提出了缓存模板攻击[34]，该攻击通过强制攻击进程与被攻击进程共享内存来实现。

该攻击方法最初由Gullasch等人提出[33]，随后扩展到针对有限责任公司（LLC）以检索加密密钥、TLS协议会话消息或跨虚拟机的键盘按键[2, 34, 35]。它还展示了其对基于AES T表的实现[36]、RSA实现[2]或 ECDSA [37]等的攻击能力。它还能检测加密库[38]。此外，Zhang等人[39]研究表明，该方法适用于多种商业PaaS云环境，可在其中实现与受害者的共存[13]。

Gruss等人[依赖于clflush指令，并遵循与Flush+Reload相同的要求3]提出了Flush+Flush攻击。该攻击旨在隐蔽性地绕过监控系统[19, 20]。与Flush+Reload的主要区别在于，该变体通过测量clflush指令的执行时间而非重载时间来恢复信息，从而避免直接访问缓存。这是避免被检测的关键事实。然而，近期研究表明该攻击仍可被检测[21, 40]。

2.2.3. Prime+探针

在虚拟环境中，共享内存并非总能通过去重技术实现，因为多数云服务商在多次攻击演示后已关闭该功能。不过，Prime+Probe [9, 41] 仍可运作，攻击者通过锁定L3缓存仍能提取敏感信息。由于Prime+Probe 不依赖系统特殊操作系统功能，几乎适用于所有虚拟化系统。

Prime+Probe 框架包含三个关键步骤：首先，攻击者会向缓存组加载已知数据；其次，触发（或等待）目标进程执行加密操作；最后，在加密完成后，攻击者会访问第一步中已知数据的位置，通过计算耗时来判断目标进程是否已清除先前加载的数据。若确认数据被清除，攻击者就能精准定位目标进程的调用路径。

该攻击最初由[1]针对L1数据缓存提出，后扩展至L1指令缓存[42]。这些方法要求攻击者与受害者共享同一核心，这降低了实用性。然而，近期研究表明其适用于LLC。研究人员已绕过多个难点以攻击LLC，例如获取其复杂的地址映射[22, 23, 43]，并成功恢复加密密钥或键盘输入的按键[11, 41, 44]。此外，Prime+Probe 攻击被用于在Amazon EC2云中恢复RSA密钥[45]。

这类攻击高度依赖精确计时器来获取目标内存访问的详细信息。如果防御系统试图限制计时器的访问权限，或通过生成噪声来掩盖计时信息，攻击成功的可能性就会降低。然而攻击者再次成功突破了这一技术难点。Prime+Abort 攻击[4]利用了英特尔硬件事务内存（TSX）的实现机制，无需依赖计时器即可获取信息。该攻击首先发起事务以初始化目标内存区域，等待后根据目标内存是否被访问而决定是否接收数据并最终中止，整个过程完全不需要计时器。

综上所述，本文提出的攻击手段以缓存为攻击目标，通过选择目标进程预期访问的内存位置实施攻击。该攻击过程包含三个阶段：初始化阶段（攻击者预先配置缓存）、等待阶段（攻击者在目标进程执行期间保持静默）以及恢复阶段（攻击者通过检查缓存状态获取目标进程信息）。这些攻击方式在具体实现方式、系统要求及可实现的攻击精度方面存在显著差异。

3. 对AES的攻击

本节将系统阐述AES加密与解密算法的核心原理。我们深入解析了被攻破的基于T表的OpenSSL AES实现机制，重点展示了针对AES-128的非访问型缓存攻击方案，其性能表现超越了此前公开的各类缓存攻击。随后，我们将说明如何将这种非访问攻击方法拓展应用于AES-256的实战攻击。我们将其命名为“逆向加密缓存攻击”，该技术通过利用加密过程中的信息获取解密轮密钥，进而转换为加密轮密钥。通过这种创新方法，攻击者能够从两个不同且连续的加密轮次中获取关键信息。

3.1. AES基础

AES算法在[46]中有所阐述。这是一种对称分组密码，以16字节的数据块为单位进行操作。加密和解密过程表现相似，它们会反复对状态（记为 S ）进行轮次变换。迭代轮数 N ，取决于密钥长度：128位使用10轮，192位使用12轮，256位使用14轮。加密过程如图1所示。每轮中，算法使用不同的轮密钥 K^r ，该密钥通过已知且固定的方案从算法密钥中获取。一旦获得轮密钥，便有直接算法可逆向推导出相同位数的加密密钥。因此，要获得256位AES密钥，至少需要获取两个连续的轮密钥。

AES将轮密钥和数据以 4×4 字节元素矩阵的形式排列。若遵循[46]中的术语，我们可以将 S^r ($0 \leq r \leq N$) 表示为轮 r 的输入状态。其每个字节元素则表示为 s_{ri} ($0 \leq i \leq 4, 0 \leq j \leq 4$)。其中 i 表示状态的行， j 表示列。我们仅使用一个下标 j 来指代状态 S_j 的列。考虑一个明文块 $p_0 p_1 p_2 \dots p_{15}$ ，算法初始状态 S_{0i0i} 的元素为 $j = p_{i+4j}$ 。

加密过程中的常规轮变换由四个步骤组成，分别标记为：SubBytes、ShiftRows、MixColumns和AddRoundKey，其中SubBytes是唯一的非线性变换。SubBytes是对状态元素应用S盒 S_{RD} 。ShiftRows是对状态元素进行转置操作，将状态 i 位置的行向左旋转。MixColumns按列对状态进行操作，该操作是将 4×4 已知矩阵与四个原始列进行矩阵乘法。AddRoundKey是对状态元素与轮密钥对应元素进行加法（在GF(2)中按位XOR）。

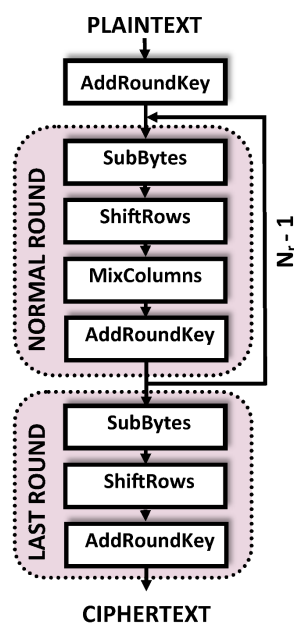


图1. AES加密过程示意图。

所有这些步骤都有一个逆步骤，用于解密过程：InvSubBytes（使用S盒 $S_R^{-1}D$ ）、InvShiftRows（向右旋转）、InvMixColumns（乘以逆矩阵）以及AddRoundKey。常规轮次变换应用于除第一轮和最后一轮之外的所有轮次。第一轮仅应用AddRoundKey步骤。最后一轮是不包含MixColumns步骤的常规轮次。

解密算法可通过逆序执行各步骤的逆运算来实现，其中InvSubBytes是除最后一轮外每轮的最后一步。在典型实现中，建议优先执行非线性步骤。在[46]中，作者提出了等效解密算法（*E quivalent Decryption Algorithm*），通过交换InvSubBytes、InvShiftRows（顺序无关）以及InvMixColumns和AddRoundKey的执行顺序，使其步骤序列与加密过程完全一致。为此，除首尾两轮外，所有轮次的轮密钥均需调整，具体方法是对其应用InvMixColumns步骤。

理解针对AES-256攻击的关键事实是，状态 S^r 可以在明文加密过程的第 r 轮或密文解密过程的第 $N_{r+1}-r$ 轮中被达到。

3.2. AES T-表实现

轮变换的步骤包括非线性S盒替换和矩阵乘法。为减少每轮运算次数并提升性能，不使用特殊汇编指令的AES软件实现会采用预计算表（T表），其中包含输入元素所有256种可能取值的对应数值。

本研究评估了OpenSSL 1.0.1f及更高版本中基于T表的实现方案，该方案在编译时启用no-asm标志。

在上述实现中，状态由四个32位变量表示，以 s 或 t 开头，每个状态列对应一个变量。变量名称在奇数轮次以 s 开头，在偶数轮次以 t 开头，将第一轮视为0。变量的最高有效字节（MSB）表示第0行的元素，最低有效字节（LSB）表示第3行的元素。加密和解密算法使用四个表，每个表包含S盒替换的结果和部分矩阵乘法。表在加密时以 Te 开头，在解密时以 Td 开头。一般而言，我们将四个表分别记为 T_i （ Te_i 和 Td_i ），其中 i 是表中所考虑的矩阵乘法列。表 T_i 包含第 i 行元素对结果列状态各行的贡献。也就是说，表格的256个位置（28）中的每一个都包含一个32位值，其中每个字节代表对每一行的贡献，对齐以执行不同贡献的相加。每个表格的大小为1024字节。列状态通过将（GF(2)中的按位XOR）不同行的贡献与轮密钥相加来计算。因此，每个输出列状态都会访问四个表格。为执行Shift-Rows步骤，使用了不同行的输入变量 s_j 的适当选择；对于目标变量 t_j ，变量 s_{i+j} 用于行 i （表格 T_i 的索引）。在本示例及整个工作中，子索引 i 和 j 中的加法均取模4。例如，在第1轮中，我们可以按如下方式获得 t_0 ：

$$t_0 = Te_0[s_0 \gg 24] \oplus Te_1[(s_1 \gg 16) \& 0xff] \oplus Te_2[(s_2 \gg 8) \& 0xff] \oplus Te_3[s_3 \& 0xff] \oplus rk[4];$$

最后一轮不包含矩阵乘法。解密算法需要一个 $S_R^{-1}D$ 表（仅InvSubBytes步骤）（表 Td_4 ）。由于MixColumns步骤的乘法矩阵中存在值为1的系数，加密算法无需该表。该表隐式存在于 Te 表中。此时， $Te_{(i+2) \bmod 4}$ 用于第 i 行，因为该行具有系数1。

3.3. 攻击AES

从前面对T表实现的描述可以推断，对每个 Te_i 的访问取决于密钥。因此，缓存命中和未命中取决于密钥。在第3.3.1节中，我们将解释如何利用缓存未命中和密文信息来检索128位密钥。

我们证明, 通过利用缓存未命中现象, 不仅能以远低于传统方法的样本量恢复128位AES密钥, 还能成功还原256位AES密钥。为此, 我们需要获取连续两轮的密钥信息。具体实施时分两步进行: 第一步针对最后一轮密钥, 第二步则针对倒数第二轮密钥。由于直接攻击倒数第二轮密钥存在技术难度, 我们通过利用加密与解密过程的等价性进行数据转换, 从而借助缓存未命中现象实现密钥恢复。

在描述这些攻击时, 我们采用与描述AES时使用的术语一致的命名法。迭代变量、子索引和超索引遵循以下规则:

i 用于引用状态或轮密钥的行。作为子索引, 它指示元素所在的行。作为迭代值, 它用于访问与某一行相关的元素 (即T表)。

j 用于迭代状态或轮密钥的列。作为子索引, 它指示元素的列。

t 是用于遍历实验中不同加密方式的迭代变量。

l 是用于遍历缓存行值的迭代变量。

r 作为超级索引, 用于标识变量 (状态、轮密钥) 的轮次。

该算法中使用的元素以大写字母表示。

S^r 表示 AES 加密算法在第 r 轮前的状态。该状态的 16 字节可通过 i 和 j 子索引访问。每次加密执行

时状态均不同, 数组 s_r 表示全部集合。

D^r 是解密算法中 S 的等效值。

K^r 表示第 r 轮的加密轮密钥。每个 16 字节可通过 i 和 j 子索引访问。

Kd^r 表示第 r 轮的解密轮密钥。该密钥可通过应用 InvMixColumns 步骤, 从对应的加密密钥 K^{Nr+1-r} 计算得出。

x_i 是关于表 Te_i 的子集的访问 (1) 或非访问 (0) 信息。该子集包含

16 个值 (64 字节), 即缓存行长度。 x_i 表示包含实验中每次加密访问信息的数组。

CK^r 是一个包含 256 个位置 (可能值) 的数组集合, 每个位置对应一个轮密钥字节。每个位置表示候选人在该具体密钥字节上收到的弃票数量。经过分析后, 最小值所在的位置 (我们称之为 arg-min 函数) 即为正确的密钥字节值。其元素可通过 i 和 j 子索引进行访问。

CKd^r 相当于解密轮密钥的 CK^r 。

3.3.1. 对 AES-128 的攻击

我们假设攻击者与受害者共享缓存, 这意味着他能够以行级粒度监控对缓存的访问。同时我们假设攻击者能够获取密文。攻击者需要从四个 T 表中逐个恢复信息。为此, 他可以监控一个表的一行数据、同时监控每个 T 表的一行数据, 甚至尝试同时监控四个表的 16 行数据 (共计 64 行)。在不同情况下, 恢复密钥所有位所需的样本数量会有所不同, 受害者观察到的攻击效果也会随之变化 [18]。同时监控的行数越多, 攻击痕迹就越明显。虽然加密时间的轻微增加可能由同一 CPU 上运行的其他程序或虚拟机造成, 但加密时间的大幅增加则会引发高度怀疑。

对于每种评估技术, 即 *Prime+Probe*、*Prime+Abort*、*Flush+Reload* 和 *Flush+Flush*, 我们采用相似流程来获取所需信息。攻击过程中遵循的流程包含三个步骤, 且在所有情况下均相同:

1. **设置:** 在发起攻击前, 攻击者必须获取 T 表在内存中的具体位置信息, 即其虚拟地址或所处的缓存组。

- 将被加载。通过这种方式，他确保缓存处于已知状态，随时准备发起攻击。
2. **测量收集：**此阶段，攻击者会监控目标表，对请求的加密操作逐一应用各种技术手段，并将收集到的信息与密文一并存储。如第2.2节所述，该步骤可分为三个阶段：初始化、等待和恢复。
3. **信息处理：**最后，攻击者利用前几步恢复的信息以及T表（监控线路内容）的信息来获取密钥。

这些主要步骤与先前的提案[36, 44]等效。其他提案也适用于我们的非访问攻击，例如Gulmezoglu等人提出的方案[47]该方法旨在检测加密的边界而非触发加密。然而，这种方法的准确性也会影响检测效果，这为评估攻击方法的有效性引入了新的变量。相比之下，前述设置可推广至所有技术，从而实现公平的比较。

下文将详细说明各算法的具体步骤及其在实验环境中的特殊性。我们设计了两种攻击场景：第一种是攻击者通过与受害者相同操作系统运行的间谍进程发起攻击；第二种是攻击者在虚拟机中运行，而受害者在另一台虚拟机中运行，两者均部署在同一台主机上。实验所用机器的详细信息见表1。需要说明的是，两种场景下的攻击步骤完全一致。

表1.实验平台详细信息。

处理机	果核	频率性	操作系统	LLC切片	有限责任公司规模	有限责任公司	VMM	访客操作系统
英特尔酷睿i5-7600K	4	3.8 千兆赫	CentOS 7.6	8	6 兆字节	12种方式	KVM	CentOS 7.6最小配置

设置

执行Flush+Reload和Flush+Flush所需的任务与Prime+Probe和Prime+Abort不同。由于Flush+Something攻击依赖共享内存，想要利用该特性的攻击者需要对受害者使用的库进行逆向工程以获取目标虚拟地址。需要注意的是，库起始地址与目标符号（表）之间的偏移量是恒定的。因此，一旦知道这个偏移量，攻击者只需将该偏移量加到库加载的虚拟地址上，就能轻松获取目标地址。相比之下，Prime+Something攻击需要目标的物理地址；更准确地说，必须知道数据将被加载的集合和切片。这些信息可以从物理地址中提取。部分物理地址可以直接从虚拟地址推断出来。事实上，指向缓存页内元素的地址位就是它们物理地址的一部分。例如，当缓存页大小为4KB时，地址的12个最低有效位在虚拟地址转换为物理地址时保持不变。然而，攻击者无法获知受害者正在处理的数据的虚拟地址和物理地址。为解决这一难题，攻击者需要创建自己的驱逐集（一组W个元素，这些元素精确映射到同一集合），并扫描整个缓存以查找受害者执行的访问操作。我们没有复现处理器的复杂地址解析功能，而是采用刘等人[原文]算法1中总结的技术，动态创建驱逐集。[41]我们还使系统中的2MB 巨页 能够与21个已知位一起工作。由于处理器的内核数是2的幂，我们只需要为每个切片获取一个集合的映射(确定数据将被放置在哪个切片中的函数只使用 标签 位)。

在分析阶段及实际攻击过程中将使用的剩余数据集，均可通过该检索到的映射关系进行构建。

测量收集

所有技术的测量收集过程都具有相似性。算法1总结了获取受害者对表访问必要信息的通用方法。这些信息将用于后续获取完整密钥。

算法1 针对T表AES实现的通用缓存攻击算法。

输入: 地址 (Te_0)、地址 (Te_1)、地址 (Te_2)、地址 (Te_3)	D T表的地址
输出: $X_0, X_1, X_2, X_3, S^{Nr+1}$	D 关于访问权限和密文的信息
1: 对于 $t = 0$ 至 加密次数	
2: 对于 $m = 0$ 至 4 时	D 初始
3: 从缓存中移除 (Te_m);	D 攻击者为攻击准备缓存
4: 端	
5:	D 等待
6: $S^{Nr+1}[t] = \text{encrypt}$ (随机明文);	D 受害方执行一次加密
7: 对于 $m = 0$ 到 4	
8: 推断受害者访问了 (Te_m)	D 恢复中
9: 如果有访问权限 (Te_m) 则 D 攻击者重新加载、刷新、探测目标或获取中止	
10: $X_m[t] = 1;$	D 攻击者决定受害者是否使用过数据。
11: 否则	
12: $\bar{X}_m[t] = 0;$	
13: 结束 if	
14: 端	
15: 为 _____ 而结束	
16: 返回 $X_0, X_1, X_2, X_3, S^{Nr+1};$	

在初始化阶段，攻击者需要将每个 Te_i 表中某一行的数据从缓存中清除。这意味着攻击者能确保在受害者使用该表前掌握其缓存状态。由于每个缓存行存储64字节，而表项每个条目仅占4字节，因此该表能存储256种可能值中的16种。随后在等待阶段，攻击者会触发加密操作。待加密完成后，在恢复阶段，攻击者会检查缓存状态。若发现缓存状态发生变化，即表明受害者很可能已使用过该T表中的数据。本研究中探讨的不同方法，主要区别在于初始化阶段清除缓存数据的方式，以及恢复被清除表行使用信息的具体方法。

在 $Flush+Something$ 的初始化阶段，目标行会通过英特尔处理器内置的 $clflush$ 指令从缓存中被清除。在 $Prime+Something$ 中，攻击者通过访问与目标行映射到相同区域的已创建清除集数据来清除目标行。英特尔实现了伪LRU清除策略，因此访问映射到该集合的 W 元素意味着缓存中的任何旧数据都将被攻击者的数据替换。在 $Prime+Abort$ 中，该操作是在使用英特尔TSX扩展定义的事务性区域内执行的。

在 $刷新+重载$ 的恢复阶段，攻击者会逐条访问被驱逐的缓存行并测量访问时间。若时间低于阈值，则表明该行曾存在于缓存中，说明算法在任意一轮中都曾访问过它。 $刷新+刷新$ 通过测量再次刷新该行所需时间来判断数据是否已被使用。刷新时间取决于缓存行的存在状态——位于缓存中的数据刷新耗时更长。 $主+探测$ 通过访问驱逐集中的数据并测量访问时间来恢复访问信息。如先前文献[41]所述，我们在探测步骤中以反向顺序访问驱逐集中的数据，即采用之字形模式读取驱逐集。若表中的数据已被加密过程使用，部分攻击者数据将被驱逐，因此“探测”该集所需时间会更长。在 $主+中止$ 阶段，由于驱逐集是在事务内被访问的，任何驱逐

攻击者从缓存中读取的数据导致程序中止。攻击者为该中止定义了一个处理程序，用于评估其原因。存在多种不同的中止原因，这些原因通过中止代码[48]进行标识。该中止代码被加载到`eax`寄存器中，并在处理程序函数中读取，以检查中止是否由驱逐操作引起。

图2显示了在重新加载步骤（图2a）和探测步骤（图2b）期间测量时间的分布。请注意，很容易建立一个阈值来区分对被监控Tc表的访问和非访问。

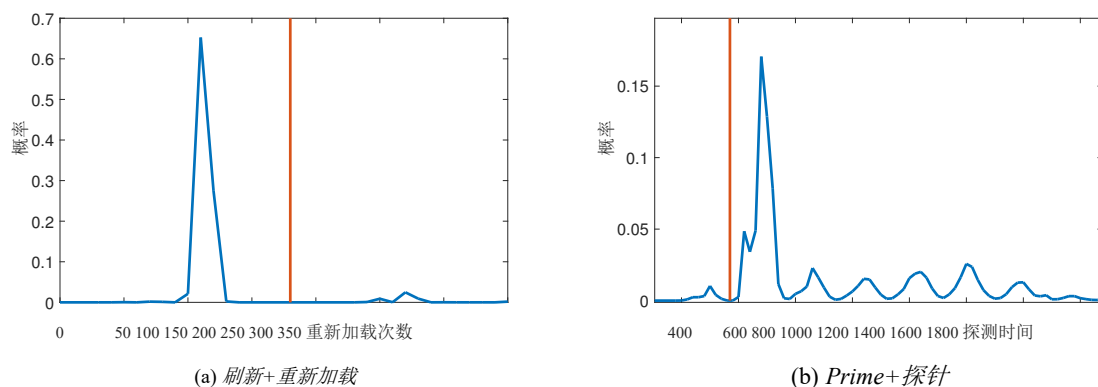


图2. 我们测试机在Reload和Probe步骤中攻击AES时获取的时序信息。阈值以红色标示。

信息处理

这一最终步骤是所有攻击的共性特征。在该阶段，我们的方法与既往攻击策略存在本质区别。传统攻击手段会检测监控缓存行中存储的数据是否被攻击目标使用，而我们的方法则反其道而行之——假设攻击目标从未使用过这些数据。该方法基于以下观察结果：

- 当检索到的信息显示受害者使用了所考虑的数据时，这些数据可能在上一轮或剩余任何一轮中被使用过。因此，每当数据在不同轮次中被使用时，该方法就会引入“噪声”。另一方面，如果被监控的缓存行在加密过程中未被访问，那么它既未在上一轮也未在其他任何一轮中被访问过。
- 代码片段的推测执行（即处理器预取其认为即将被使用的数据）可能导致攻击者在缓存中发现受害者未使用的数据（误报）。但要通过非访问方式获得误报，这些数据必须在加密的最后一轮使用，并在加密结束后的短时间内被其他进程清除，且需在恢复步骤之前完成。
- 每个非访问样本会通过舍弃最多 $n/4$ 个键值来缩小密钥搜索空间（ n 等于缓存行中存储的16个表项，这些表项用于获取4个输出字节）。因此，我们能从每个样本中获取信息。相比之下，在考虑访问时，只有当某个候选值显著优于其他值时，所获取的信息才对密钥检索具有实际价值。
- 通过这种方式获取的信息更可能适用于整个加密过程的所有轮次，或至少适用于倒数第二轮次，因为它涉及整个加密过程，因此我们可以利用这些信息来恢复更长的密钥（192位或256位）。

我们在第3.2节中看到，每个表Tc在每轮中会被访问四次。因此，加密过程中未访问具体缓存行的概率由公式(1)给出，其中 n 表示缓存行可容纳的表项数量， N_r 表示算法的轮数。特别地，在我们的情况下，当缓存行大小为64字节时， n 为16（每个表项的

对于32位的AES-128算法，其轮数（ N_r ）为10。因此，未访问缓存行的概率为7.5%。这意味着每进行14次加密操作中，就有1次能提供有效信息。

$$Pr[no\ access\ Te_i] = \left(1 - \frac{n}{256}\right)^{N_r * 4} \quad (1)$$

如前所述，我们关注加密过程的最后一轮。该轮包括SubBytes、ShiftRows和AddRoundKey操作。参考最后一轮的输入状态和T表，我们可以用 $s_{Ni}, j^r + 1 = K_i^N, j^r \oplus Te_{i+2[s_{Ni}, j^r + i]}$ 来表示输出元素。当缓存行在加密过程中未被使用时，我们可以丢弃所有键值——对于给定的输出字节，这些键值本应访问上述缓存行中存储的Te表条目。例如，给定密文字节“0F”和缓存行“01”、“AF”、“B4”和“29”，若该行在加密后仍保持未使用状态，则丢弃键字节值 $0F \oplus 01$ 、 $0F \oplus AF$ 、 $0F \oplus B4$ 和 $0F \oplus 29$ 。如第3.2节所示，同一张表用于获取输出状态行的所有元素。这意味着每个Te表也会独立地丢弃具有相同*i*值的 K_i, j 值，与*j*索引无关。

最后一轮密钥恢复算法的总体描述见算法2。最后一轮的密钥字节 K_{iN}, j^r 从输出状态 S^{Nr+1} （密文）和存储在 xi 中、通过算法1恢复的各表访问信息 Te_i 中获取。输出状态（ S^{Nr+1} ）通过将已知密文向量按矩阵排列获得，如第3.1节所述。该算法首先为密钥的16个字节（ K_i, j ）中的每个初始化一个256元素数组。该数组（ CK_{Ni}, j^r ）将包含每个密钥字节候选的丢弃票数。负票数较少的候选即为密钥值。

算法2 关键字字节 K_{iN}, j^r 的恢复算法。

输入： $X_0, X_1, X_2, X_3, S^{Nr+1}$ D 上一步收集的访问和密文信息。

输出： K_{Ni}, j^r D 秘密密钥值

1: 对于 $l = 0$ 到 256

2: $CK_{i,j}^{Nr}[l] = 0;$

D 候选人数组的初始化

3: 端

4: 对于 $t = 0$ 至 加密次数

5: 如果 $xi + 2[t] == 0$ 则

6: 对于 $l = 0$ 至 n 时 Dn 表示缓存行所包含的条目数量

7: $CK_{i,j}^{Nr}[S_{i,j}^{Nr+1}[t] \oplus Te_{i+2}[l]] ++;$ ▷ 投票淘汰关键候选人

8: 端

9: 结束 if

10: 端

11: 返回 $\text{argmin}(CK_{Ni}, j^r)$; D 投票最少的候选者将被舍弃，即为 **密钥**。

图3展示了16个 CK_{Ni}, j^r 数组中某个值的分布情况，该数组记录了攻击过程中每个可能密钥值在特定字节密钥下收到的被丢弃投票次数。横轴表示所有可能的密钥值，纵轴表示每个选项被丢弃的次数。最小值与其他可能密钥值明显区分，并以红色高亮显示。该最小值对应的索引即为密钥。此方法在最佳情况下平均仅需不到3000次加密即可恢复密钥的128位。我们将在第4节展示更多结果并提供详细说明。

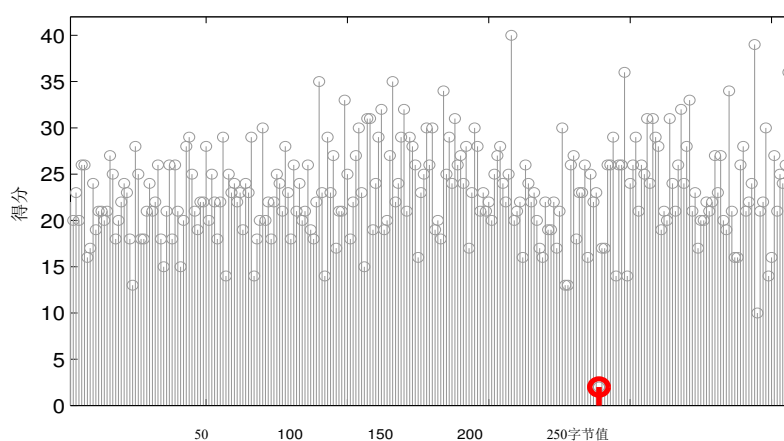


图3 关键字节可能值及其对应的丢弃分数。

3.3.2. 对AES-256的攻击

针对AES-256的攻击是之前所述加密最后一轮攻击的延伸。与前文所述情况相同，本次攻击仅需满足相同的前提条件。两种攻击的设置步骤和测量收集步骤完全一致，唯一区别在于信息处理阶段增加了额外步骤。我们首先使用算法2获取密钥的128位，然后利用已猜测的128位密钥及已使用信息（非访问记录和密文）推导出剩余的128位未知密钥。类似地，我们通过分析未使用的缓存行信息来逐步筛选密钥值，直至获得完整密钥。这种操作之所以可行，是因为这些信息涉及整个加密过程——正如前文所述，若某条缓存行在加密过程中未被访问，说明其在所有轮次中均未被调用。

需要注意的是，执行AES-256加密时，不访问某一行的概率（该概率与轮数相关，由公式1给出）为2.69%（14轮）。因此，攻击AES-256需要更多样本才能成功猜中完整密钥。在这种情况下，约1/37的样本包含有效信息。这意味着我们需要收集的有用样本数量至少与前一种情况相同。

理解该攻击的关键在于理解加密与解密之间的等价性。通过解密算法，我们对加密过程中的数据进行转换，使其能够轻易推导出轮密钥，而无需考虑其可能的取值。因此，我们将这种方法称为加密-解密攻击（EBD）。

通过分析AES加密过程可以发现，每轮加密的输出结果都会作为索引，在执行移行操作后用于访问下一轮的T表。考虑到未使用缓存行所蕴含的宝贵信息，我们能够推断出在倒数第二轮之前无法获取的状态值，即 SN_{r-1} 。换言之，如果某条缓存行未被访问，那么所有可能导致该行被访问的状态值都可以被舍弃。

当我们掌握最后一轮密钥后，仅需考虑加密函数即可轻松推导出该轮次的输入（ SN_{r-1} ）。但由于除初始轮次外所有后续轮次均执行混列操作，仅凭对某表缓存行未被使用的确定性及加密函数知识，仍无法获取这些轮次的密钥信息。不过，若运用解密函数并结合加密与解密的等价性，我们可通过数

据转换推导出解密函数的状态信息——这与加密函数中的 SN_{r-1} 状态具有等效性。

图4用虚线标出了等效解密和加密算法具有相同值的时刻。根据该图，我们通过SubBytes和Shift-Rows步骤，建立了加密状态 S_r 与解密状态 DN_{r+1-r} 之间的双向关系。

这些关系在公式(2a)和(2b)中定义。这些关系以及状态 S_{Nr-1} 中不可能取值的知识,使我们能够确定状态 D_2 中不可能取的值。由于我们所称的中间值 IV^1 的值可轻易从解密函数中获得,因此我们的场景与前述情况相似($D_{2i}, j = IV^1_i, j \notin CKd_{1i}, j$)。

$$D^{Nr+1-r} = \text{ShiftRows}(S_{RD}(S^r)) \quad (2a)$$

$$S^r = S_{RD}(\text{InvShiftRows}(D^{Nr+1-r})) \quad (2b)$$

算法3展示了如何获取解密算法第一轮(第二轮)的正确候选轮密钥。第一轮解密的密钥与加密最后一轮使用的密钥完全相同,该密钥先前通过针对最后一轮的非访问缓存攻击(算法2)获得。由于我们已知输入数据(密文)和轮密钥,可以执行解密算法的第一轮并获取第一轮状态 D_1 。将Td表应用于该状态后,我们计算出第一轮的中值 IV^1 。此时若执行解密算法的下一步AddRoundKey操作,结果将为 D_2 ,该值可转换为加密 S_{13} 状态,如公式(2b)所示。

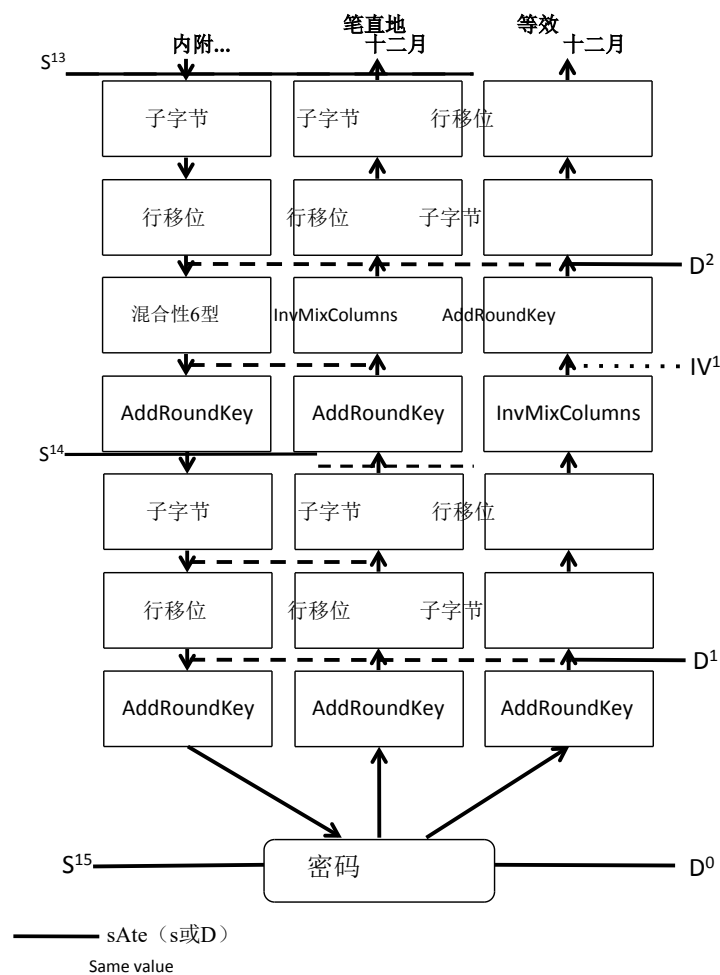


图4. 加密、直接解密和等效解密之间的关系。

在T表实现中,状态(包括AES256的 S_{Nr-1} 或 S_{13})被用作索引来访问Te表。需要注意的是,未访问表中某条记录的缓存行也表明了哪些表索引未被使用,这类似于指示哪些状态值可以被舍弃。随后,我们应用公式(2a)将状态 S_{Nr-1} 的舍弃值转换为状态 D_2 的舍弃值。经过此转换后,我们获得了关于

在解密序列的第一轮中添加RoundKey。我们可以投票舍弃解密轮密钥的可能值。投票支持那些在解密状态转换为加密状态后，会导致对未使用线路进行内存访问的值。

每个字节中投票数最少的值即为解密轮密钥。加密密钥与解密密钥存在关联性，通过应用InvMixColumns操作（或根据转换方向选择等效操作）可相互推导。经过这种信息处理，我们就能获得两个连续的轮密钥，也就是完整的AES-256密钥。由于缓存信息在整个加密过程中都适用，该算法可递归应用于整个加密/解密流程。

通过针对AES-256的这种加密-解密缓存攻击，我们在最佳情况下仅需不到10,000个样本即可获取密钥。更多细节将在第4节中提供。

算法3 关键字字节 $Kd_{1i, j}$ 的恢复算法。

输入: $X_0, X_1, X_2, X_3, S^{N_r+1}$ 和 $K_{N^r}D$ 关于访问、密文和最后一轮密钥的信息。
输出: $Kd_{1i, j}$ D 秘密解密密钥值

- 1: $Kd^0 = K^{N_r}$
- 2: 对于 $l = 0$ 到 256
- 3: $CKd_{ij}^1[l] = 0$; D 候选人数组的初始化
- 4: 端
- 5: 对于 $t = 0$ 至 加密次数
- 6: 如果 $x_i[t] == 0$ 则
- 7: $D^0 = S^{N_r+1}[t]$;
- 8: $D^1 = D^0 \oplus Kd^0$;
- 9: 对于 $j = 0$ 到 4
- 10: $IV^1 j = Td_0[D^{10}, j] \oplus Td_1[D^{11}, j-1] \oplus Td_2[D^{12}, j-2] \oplus Td_3[D^{13}, j-3]$ D 获取中间值
- 11: 端
- 12: 对于 $l = 0$ 至 n 时 Dn 表示缓存行所包含的条目数量
- 13: $CKd_{1i, j} = [IV^1 ij \oplus S_{RD}[l]]++$; $\triangleright$ 投票淘汰关键候选人
- 14: 端
- 15: 结束 if
- 16: 端
- 17: 返回 $\text{argmin}(CKd_{1i, j})$; D 投票最少的候选者将被舍弃，即为 **密钥**。

4. 结果讨论

我们的实验配置包含一台英特尔酷睿i5-7600K处理器（3.80 GHz），配备4个CPU核心，两个32 KB的专用L1缓存（分别用于数据和指令缓存），256KB的L2缓存，以及6MB的L3缓存（所有核心共享，行大小为64字节）。系统安装了CentOS 7.6，实验用虚拟机采用CentOS 7.6精简版，均配置一个虚拟核心。虚拟机管理程序为WAS KVM。相关技术参数详见表1。本次实验以OpenSSL 1.0.1f版本为测试对象，该版本包含基于T表实现的AES加密算法。

在上一节中，我们介绍了适用于不同长度AES密钥（128位或256位）的攻击方法。这些攻击假设攻击者已知未使用的缓存行以丢弃可能的密钥值，并且已知密文。我们在两种不同场景（虚拟化与非虚拟化环境）下进行了实验，考虑了获取缓存利用率信息的不同技术（Prime+Probe、Prime+Abort、Flush+Reload和Flush+Flush）以及不同的密钥长度。表2显示了在非虚拟化环境中（受害者和攻击者进程运行于

（相同操作系统）。类似地，表3展示了虚拟化环境下的结果（受害者运行于一个虚拟机中，攻击者运行于另一个共享同一主机的虚拟机中）。

表2 非虚拟化环境中检索完整密钥所需的平均样本数量。

	刷新+重新加载	冲洗+冲洗	Prime+探针	Prime+中止
AES 128	3000	15,000	14,000	3500
AES 256	8000	35,000	38,000	8000

表3 虚拟化环境中检索完整密钥所需的平均样本数量（跨虚拟机）

	刷新+重新加载	冲洗+冲洗	Prime+探针
AES 128	10,000	40,000	45,000
AES 256	28,000	100,000	110,000

在应用Prime+Abort技术时，我们需要监控四个不同的数据流。为此必须划分四个事务处理区域进行区分，这也意味着需要消耗更多资源。需要说明的是，我们未在虚拟化环境中测试Prime+Abort技术的运行效果。由于TSX指令已实现虚拟化，理论上应在跨虚拟机场景中支持Prime+Abort技术。但实验设置中攻击者准备缓存、触发加密并等待结果的流程导致异常终止事件过多。由于这些异常终止事件成因相似，难以区分加密导致的异常与噪声干扰。鉴于此，且考虑到研究结论已初步验证，我们未对跨虚拟机场景下采用Prime+Abort技术采集的数据进行评估。

为公平比较访问方法与非访问方法，我们还检索了128位密钥的访问方法密钥。表4展示了两种方法的结果，以及成功检索密钥所需追踪次数的改进百分比。所有案例中，样本数量均显著减少。

表4 采用访问法（第一列）、本方法（第二列）及改进后本方法获取完整128位密钥所需的平均样本数。(V)虚拟化场景下的平均值。

	访问	无访问权	改善
刷新+重新加载	100,000	3000	97%
冲洗+冲洗	250,000	16,000	94%
Prime+探针	190,000	14,000	93%
Prime+中止	110,000	3500	97%
刷新+重新加载(V)	400,000	10,000	98%
刷新+刷新(V)	1,120,000	40,000	96%
Prime+探针(V)	1,250,000	45,000	96%

我们注意到，当同时监控多条数据线时，每个表格提供的信息量存在差异。这一现象在图5中清晰呈现：图中通过颜色映射展示了候选密钥数组中每个字节的分布情况——深色区域表示被舍弃的正确密钥值较少，浅色区域则对应被舍弃的密钥值较多。当输入明文均匀分布时，随着加密次数的增加，最低得分的差异愈发显著。图中显示每个字节都与同一表格存在关联模式。我们发现每个表格获取的有效样本数量各不相同，这源于预取器机制和我们采用的均匀访问模式可能引发的连锁反应。这意味着需要进行更多次加密操作才能获取完整密钥。

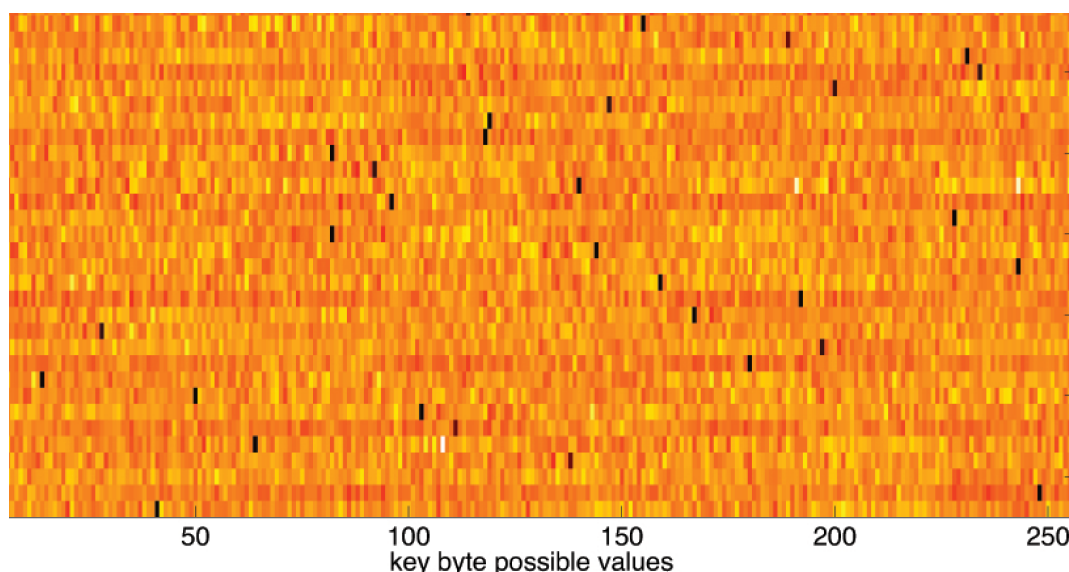


图5 为采用Flush+Reload技术检索到的32字节密钥丢弃分数分布的色阶图，正确密钥值对应最深色区域。

我们通过 EBD 攻击完全恢复了完整的AES-256密钥。对比获取128位密钥与256位密钥所需的追踪次数，采用我们方法攻击AES-256的难度仅为三倍。未访问某行的概率差异解释了实验结果：AES-256为2.69%，AES-128为7.5%，理论上这意味着获取相同信息需要2.8倍的加密量。实际测试中，我们测得AES-128及其各行在刷新+重载模式下的缓存未命中率分别为5.29%、3.87%、4.55%和4.31%，而AES-256则为1.52%、0.93%、1.31%和1.02%。这些结果支持了文献[49]的结论：虽然密钥长度增加会导致暴力破解复杂度呈指数级增长，但这一规律不适用于侧信道攻击。此外，通过使用加密算法的最后两轮（该版本采用不同表格计算加密的最后轮），他们得出结论：与基于缓存的AES-128攻击相比，AES-256仅增加了6-7倍的复杂度。而我们则证明，这一系数等于3。

在 EBD 攻击案例中，我们未发现可对比的类似结果或实验设置。所有研究的攻击均可归类为高度理论性。文献中，Acicmez等人[50]指出其攻击方法可应用于加密的第二轮，但未提供实验结果。2009年，Biryukov等人[51]提出了一种针对AES-256的区分式相关密钥攻击，其数据复杂度为 2^{131} 。随后，他们略微改进了结果[52]，实现了 2^{119} 的数据复杂度。在[53]中，他们描述了一种使用两个相关密钥恢复256位密钥的攻击，但其目标密码是九轮版AES-256且使用了选择明文。另一项攻击[54]在相关子密钥场景下运行，使用四个相关密钥针对13轮AES-256；其具有 2^{76} 的时间和数据复杂度。Bogdanov等人[55]提出了一种基于双子图的密码分析技术，该技术同样导致了密钥恢复的非实际复杂度。Kim[56]还从理论上阐述了如何应用差分故障分析来减少AES-192和AES-256所需的故障数量。

如前所述，每个包含T表未访问信息的检索样本都会缩小密钥搜索空间。这意味着，如果攻击在未完成时因故中断，攻击者决定采用暴力破解方式获取密钥，那么需要验证的密钥数量将大幅减少。图6展示了密钥搜索空间的二进制对数与检索样本总数（即每次加密后剩余未知位数）的关系。值得注意的是，在使用256位密钥的加密场景中，初始阶段仍有128位未知；如前文所述，我们通过两轮运算即可获得完整密钥，且使用相同数据进行计算。

每轮可恢复128位信息。因此，该数值表示每轮未知位数的平均值。

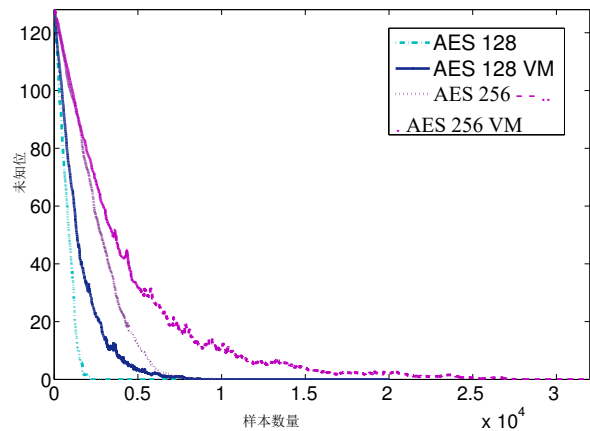


图6 Flush+Reload 技术每次加密后仍未知的关键位。

为将这些数据置于上下文中，我们还测量了两个过程（受害者和攻击者）需要相互作用直至获得完整密钥所需的时间。由于信息处理是离线进行的，这一时间被称为测量收集步骤。表5显示了这些平均时间。

表5收集图6数据所需平均时间。

攻击持续时间	
AES-128	43毫秒
AES-128 (VM)	558毫秒
AES-256	135毫秒
AES-256 (VM)	1732毫秒

假设采集样本数量随时间呈线性变化，这些结果表明，若要在此类攻击成功前检测到它们，必须迅速采取行动。检测时间需达到毫秒级。Chiappetta等人[19] 让受害者运行100毫秒，这意味着我们的方法会在他们察觉攻击前成功。即使检测时间约为10毫秒，也意味着大约一半的密钥会被泄露。在CloudRadar[20]中，检测时间约为5毫秒，此时虚拟机管理程序需要迁移机器。CacheShield[21]会在4.48毫秒，此时攻击者必须立即触发防御措施。可以合理推测，即便启用任何防御手段，攻击者与受害者仍能保持8-10毫秒的交互时间。这意味着部分密钥信息仍会泄露，而采用访问控制方案则不会发生信息泄露。需要特别说明的是，我们假设受害者正在加密随机明文。若攻击者能迫使受害者以特定方式加密选定明文，导致未被访问的代码行数量增加，我们的防御方案将能更快生效且几乎不会触发缓存未命中，从而使原有防御手段失效。

我们证明，在搭载全相联缓存的英特尔处理器中，采用非访问式攻击方法所需的样本量比传统访问式方法更少，即可成功获取AES密钥。然而，对于易受缓存攻击的AMD和ARM处理器[10 – 12]，我们的攻击方法同样适用。我们相信在这些处理器上，成功攻击所需的样本量也能实现类似缩减。近期研究还表明，非全相联缓存同样存在缓存攻击风险[57]。因此，我们的方法对非全相联缓存同样可行。只要攻击者能够将目标数据从缓存中驱逐，就能利用我们的方法提升针对AES的攻击效率。

T表加密方案已被软件中的S盒加密方案取代。在这种情况下，加密过程是通过一个存储256字节值的唯一表格实现的。攻击者也可以利用我们的方法来针对这种实现方式。如果攻击者能够清空加密最后一轮中的某一行数据，并且考虑到每轮对表格的16次访问意味着1%的未访问概率（攻击是可行的），那么每条未被访问的行将使我们能够从每个密钥字节的256个可能值中丢弃64个值。需要指出的是，我们针对T表实现提出的攻击方案会丢弃密钥4个字节中256个可能值中的16个。因此，理论攻击对新实现最后一轮的估计泄露量约为针对T表实现所提出攻击的75倍。

5. 结论

云计算的广泛应用带来了诸多优势，但也引发了新的安全与隐私风险。其中，虚拟机间资源共享（特别是共享缓存内存）导致的信息泄露问题尤为突出。即便在非虚拟化环境中，所有运行进程共享缓存时，这种风险同样存在。

本研究提出了一种利用现有缓存攻击获取信息的新方法：*刷新+重载*、*刷新+刷新*、*预取+探测*和*预取+中止*。我们聚焦于未被使用的缓存行，并以AES T表实现为验证对象。尽管已知T表版本的AES算法易受侧信道攻击，但在新版OpenSSL中仍可使用（通过no-asm标志编译）。通过非访问攻击，我们证明了获取AES-128密钥所需加密量可减少93%至98%。这一改进对开发针对真实服务器的实用攻击至关重要，因为AES密钥通常为会话密钥，需通过密钥交换算法定期更新。此外，我们证明若存在检测缓存攻击的系统，泄露信息量将取决于检测时间，而我们的方法可从攻击初始阶段就恢复信息。

此外，我们针对256位AES算法提出了一种兼具时间复杂度与计算复杂度优势的实用攻击方案——加密解密（EBD）缓存攻击。我们在真实实验环境中成功实施该攻击，最佳情况下仅需10,000次加密操作，便能在毫秒级时间内获取加密密钥。EBD通过将攻击过程分解为两个128位阶段来实现密钥提取：第一阶段利用未被访问的密钥行，通过丢弃未使用数据的密钥值来获取密钥的最后128位；第二阶段则将第一阶段获得的128位子密钥输入解密函数（执行部分AES解密操作），从而进入相当于加密状态的计算环境，此时未访问的密钥行可被用于获取剩余128位密钥。实验结果表明，AES-256的攻击难度仅为AES-128的三倍。

作者贡献： 概念设计：S.B.、P.M.、J.-M.d.G. 和 J.M.M.；数据管理：S.B.；资金获取：S.B. 和 J.M.M.；研究调查：S.B.；方法学：S.B. 和 P.M.；软件开发：S.B.；监督：J.M.M.；初稿撰写：S.B. 和 P.M.；审阅与编辑：J.-M.d.G.

资助： 本研究由西班牙经济与竞争力部资助，资助编号为TIN-2015-65277-R、AYA2015-65973-C3-3-R及RTC-2016-5434-8。

利益冲突： 作者声明无利益冲突。

参考文献

1. Osvik, D. A.; Shamir, A.; Tromer, E. 缓存攻击与对策：以AES为例。收录于《密码学专题会议论文集——CT-RSA》，美国加利福尼亚州圣何塞，2006年2月13-17日；施普林格出版社：德国海德堡/柏林，2006年；第3860卷，第1-20页。
2. Yarom, Y.; Falkner, K. flush+reload: 一种高分辨率、低噪声的L3缓存侧信道攻击。USENIX 安全研讨会论文集，美国加利福尼亚州圣地亚哥，2006年8月20-22日；USENIX 协会：美国加利福尼亚州伯克利，2014年；第719-732页。

3. Gruss, D.; Maurice, C.; Wagner, K. Flush+Flush: 一种更隐蔽的最后级缓存攻击。 *CoRR* **2015**, _14. [[CrossRef](#)]
4. Disselkoe, C.; Kohlbrenner, D.; Porter, L.; Tullsen, D. 《Prime+Abort: 一种利用Intel TSX 的无定时器高精度L3缓存攻击》。收录于第26届 USENIX 安全研讨会论文集 (USENIX Security17), 加拿大不列颠哥伦比亚省温哥华, 2017年8月16-18日; 美国 USENIX 协会: 加利福尼亚州伯克利, 2017年; 第51-67页。
5. Bernstein, D.J. AES的缓存定时攻击2005年。在线获取: <http://palms.ee.princeton.edu/system/files/Cache-timing+attacks+on+AES.pdf> (2019年2月28日访问)。
6. Weiß, M.; 海因茨, B.; 施图姆普夫, F. 虚拟化环境中AES的缓存定时攻击。载于《国际金融密码学与数据安全会议论文集》——FC, 2012年2月27日至3月2日, 博奈尔岛克拉伦迪克; 施普林格出版社: 德国海德堡/柏林, 2012年; 第7398卷, 第314–328页。
7. 布鲁姆利(B.B.)、哈卡拉(R.M.)合著的《缓存定时模板攻击》一文, 收录于《密码学进展会议论文集——asia-crypt》, 2009年12月6日至10日于日本东京举行; 该文由德国施普林格出版社于2009年在海德堡/柏林出版, 编号为第5912卷, 页码667-684。
8. Chen, C.; Wang, T.; Kou, Y.; Chen, X.; Li, X. 改进基于跟踪驱动的I-Cache时序攻击对{RSA}算法的攻击。 *系统软件杂志* **2013**, *86*, 100–107. [[交叉引用](#)]
9. Inci, M.S.; Gulmezoglu, B.; Irazoqui, G.; Eisenbarth, T.; Sunar, B. 说真的, 别再让我心烦了! 公共云中的跨虚拟机RSA密钥恢复。密码学电子预印本档案, 报告2015/898. 2015年。在线获取: <http://eprint.iacr.org/> (2019年2月28日访问)。
10. Irazoqui, G.; Eisenbarth, T.; Sunar, B. 交叉处理器缓存攻击。收录于第11届ACM亚洲计算机与通信安全会议 (Asia China化学会 '16) 论文集, 中国西安, 2016年5月30日至6月3日; ACM: 美国纽约州纽约市, 2016年; 第353–364页。
11. Lipp, M.; Gruss, D.; Spreitzer, R.; Mangard, S. ARMageddon: Last-Level Cache Attacks on Mobile Devices. *arXiv* **2015**, arXiv:1511.04897.
12. 格林, M.; 罗德里格斯-利马, L.; 赞克尔, A.; 伊拉索基, G.; 海斯尔, J.; 艾森巴特, T. 《AutoLock: 为何ARM架构的缓存攻击比想象中更难》。收录于第26届 USENIX 安全研讨会论文集 (USENIX 安全17), 加拿大不列颠哥伦比亚省温哥华, 2017年8月16-18日; USENIX 协会: 美国加利福尼亚州伯克利, 2017年; 第1075-1091页。
13. Ristenpart, T.; Tromer, E.; Shacham, H.; Savage, S. Hey, you, get off my cloud: exploring information leakage in third-party compute clouds. In Proceedings of the ACM Conference on Computer and Communications Security (<Chinese化学会>), Chicago, IL, USA, 9–13 November 2009; ACM: New York, NY, USA, 2009; pp. 199–212.
14. 张Y、朱尔斯A、奥普雷A、赖特M.K. 《居家独居: 基于侧信道分析的云环境共居检测》。收录于《IE-EE安全与隐私研讨会论文集》, 美国加州伯克利市, 2011年5月22-25日, 第313-328页。
15. 王Z、李R.B. 《新型缓存设计对抗软件缓存侧信道攻击》。收录于《第34届国际计算机体系结构年会论文集》, ISCA '07, 美国加利福尼亚州圣地亚哥, 2007年6月9-13日; ACM出版社: 美国纽约, 2007年, 第494-505页。
16. Kim, T.; Peinado, M.; Mainar-Ruiz, G. stealthmem: 云环境中的系统级缓存侧信道攻击防护。收录于《第21届 USENIX 安全研讨会论文集》 (USENIX Security12), 美国华盛顿州贝尔维尤, 2012年8月8-10日; USENIX 协会: 美国加利福尼亚州伯克利, 2012年; 第189-204页。
17. Liu, F.; Ge, Q.; Yarom, Y.; Mckeen, F.; Rozas, C.; Heiser, G.; Lee, R.B. 催化剂: 在云计算中抵御最后一级缓存侧信道攻击。收录于《2016年IEEE高性能计算机体系结构国际研讨会论文集》 (HPCA), 西班牙巴塞罗那, 2016年3月12-16日, 第406-418页。
18. Briongos, S.; Malagon, P.; Risco-Martin, J.L.; Moya, J.M. 对AES侧信道缓存攻击的建模。收录于《2016年夏季计算机模拟会议论文集》, SCSC '16, 加拿大蒙特利尔, 2016年7月24-27日; 国际计算机模拟学会: 美国加利福尼亚州圣地亚哥, 2016年; 第37卷: 1-37页。
19. Chiappetta, M.; Savas, E.; Yilmaz, C. 基于硬件性能计数器的缓存侧信道攻击实时检测. *应用软件计算* **2016**, *49*, 1162–1174. [[交叉引用](#)]
20. Zhang, T.; Zhang, Y.; Lee, R.B. CloudRadar: 云中的实时 Time Side-Channel Attack Detection 系统。第19届国际攻击、入侵与防御研究研讨会 In Proceedings, RAID2016, 法国巴黎, 2016年9月19日至21日; Monrose, F., Dacier, M., Blanc, G., Garcia-Alfaro, J.编; 施普林格国际出版: 瑞士沙姆, 2016年; 第118-140页。

21. Briongos, S.; Irazoqui, G.; Malagon, P.; Eisenbarth, T. CacheShield: 通过自我观察检测缓存攻击。收录于第八届ACM数据与应用安全及隐私会议论文集 (codaspy '18), 美国亚利桑那州坦佩市, 2018年3月19-21日; ACM出版社: 美国纽约州纽约市, 2018年; 第224-235页。
22. 莫里斯 (C.)、勒斯库阿尔内克 (N.)、诺伊曼 (C.)、希恩 (O.)、弗朗西永 (A.) 合著的《基于性能计数器的Intel最后一级缓存复杂地址反向工程》, 收录于《攻击、入侵与防御研究会议论文集》(2015年11月2-4日, 日本京都), 由博斯 (H.)、蒙罗斯 (F.)、布兰克 (G.) 主编, 施普林格国际出版社 (瑞士夏姆) 2015年出版, 第48-65页。
23. Irazoqui, G.; Eisenbarth, T.; Sunar, B. Systematic Reverse Cache Sli 处理器选型工程。收录于《2015年欧洲微系统设计会议论文集》, DSD 2015, 葡萄牙马德拉, 2015年8月26-28日, 第629-636页。
24. Arcangeli, A.; Eidus, I.; Wright, C. 利用 KSM 提高存储密度。载于《Linux研讨会论文集》, 加拿大魁北克省蒙特利尔, 2009年7月13-17日; 第19-28页。
25. 内核同页合并。2015年。在线可用: http://kernelnewbies.org/Linux_2_6_32#head-d3f32e41df508090810388a57efce73f52660ccb/ (访问日期: 2019年2月28日)。
26. 胡W.《格调度与隐蔽信道》。收录于《IEEE安全与隐私研究研讨会论文集》, 美国加利福尼亚州奥克兰, 1992年5月4-6日; IEEE计算机学会: 美国华盛顿特区, 1992年; 第52-61页。
27. 科赫尔 (P.C. Kocher): 《针对Diffie-Hellman、RSA、DSS等系统实现的定时攻击研究》。收录于《密码学进展会议论文集》(1996年8月18-22日, 美国加利福尼亚州圣巴巴拉市), 由德国施普林格出版社 (海德堡/柏林) 出版, 第1109卷, 第104-113页。
28. Kelsey, J.; Schneier, B.; Wagner, D.; Hall, C. 乘积密码的侧信道密码分析. *计算机安全杂志* 2000, 8, 141-158. [交叉引用]
29. Page, D. 缓存内存作为密码分析侧信道的理论应用。密码学电子预印本档案库, 报告2002/169。2002年。在线获取: <http://eprint.iacr.org/> (2019年2月28日访问)。
30. 津野Y、斋藤T、铃木T、志贺M、宫内H.《基于缓存计算机的DES密码分析》。收录于《密码硬件与嵌入式系统会议论文集——CHES, 德国科隆, 2003年9月8-10日》; 施普林格出版社: 德国海德堡/柏林, 2003年; 第2279卷, 第62-76页。
31. Irazoqui, G.; Inci, M.S.; Eisenbarth, T.; Sunar, B. 细粒度跨虚拟机攻击Xen和V Mware Are 可能! 发表于《2014年IEEE第四届大数据与云计算国际会议论文集》(bdcloud '14), 2014年12月3日至5日, 澳大利亚悉尼; 美国华盛顿特区IEEE计算机学会, 2014年, 第737-744页。
32. Percival, C. 缓存缺失: 趣味与专业。《BSDCan》2005年 In Proceedings, 加拿大安大略省渥太华, 2005年5月13-14日。
33. Gullasch, D.; Bangerter, E.; Krenn, S. 缓存游戏——将基于访问的AES缓存攻击付诸实践。收录于《IEEE安全与隐私研讨会论文集》, S&P, 美国加利福尼亚州伯克利, 2011年5月22-25日; 第490-505页。
34. 格鲁斯, D.; 斯普雷策, R.; 曼加德, S.《高速缓存模板攻击: 对全相联最后一级高速缓存的自动化攻击》。收录于《USENIX 安全研讨会论文集》, 美国华盛顿特区, 2015年8月12-14日; USENIX 协会: 美国加州伯克利, 2015年; 第897-912页。
35. Irazoqui, G.; Inci, M.S.; Eisenbarth, T.; Sunar, B.《幸运13号反击》。收录于第十届ACM信息、计算机与通信安全研讨会论文集, 中国化学学会 '15, 新加坡, 2015年4月14-17日; ACM: 美国纽约州纽约市, 2015年; 第85-96页。
36. Irazoqui, G.; Inci, M.S.; Eisenbarth, T.; Sunar, B. 等.《稍等! 一种针对AES的快速跨虚拟机攻击》. 收录于《攻击、入侵与防御研究研讨会论文集》, RAID, 瑞典哥德堡, 2014年9月17-19日; 施普林格出版社: 德国海德堡/柏林, 2014年; 第8688卷, 第299-319页。
37. Yarom, Y.; Benger, N. 利用 flush+reload 缓存侧信道攻击恢复 OpenSSL ECDSA 非对称密钥; 密码学电子预印本档案库, 报告 2014/140。2014年。在线获取: <http://eprint.iacr.org/> (2019年2月28日访问)。
38. Irazoqui, G.; Inci, M.S.; Eisenbarth, T.; Sunar, B.《知邻: 云环境中的加密库检测》。《隐私增强技术会议论文集》2015年, 美国宾夕法尼亚州费城, 2015年6月30日至7月2日; 第25-40页。

39. Zhang, Y.; Juels, A.; Reiter, M.K.; Ristenpart, T. PaaS云中的跨 Tenant Side Channel Attacks。收录于2014年 ACM SIGSAC 计算机与通信安全会议论文集, 中国化学会 '14, 美国亚利桑那州斯科茨代尔, 2014年11月3-7日; ACM: 纽约, 美国, 2014年; 第990-1003页。
40. Kulah, Y.; Dincer, B.; Yilmaz, C.; Savas, E. SpyDetector: 一种在运行时检测侧信道攻击的方法。《国际信息安全杂志》2018。[交叉引用]
41. 刘F、亚罗姆Y、葛Q、海瑟G、李R.B.《最后一级缓存侧信道攻击具有实用性》。收录于《IEEE安全与隐私研讨会 (S&P) 论文集》, 美国加利福尼亚州圣何塞, 2015年5月17-21日, 第605-622页。
42. Aciicmez, O.; Schindler, W. RSA实现中因指令缓存分析导致的漏洞及其在OpenSSL上的演示。收录于《密码学专题会议论文集——CT-RSA 2008》, 美国加利福尼亚州旧金山, 2008年4月8-11日; 施普林格出版社: 德国海德堡/柏林, 2008年; 第256-273页。
43. Yarom, Y.; Ge, Q.; Liu, F.; Lee, R.B.; Heiser, G. Intel 最终级缓存映射研究. 加密学电子预印本档案, 报告 2015/905. 2015. 在线获取地址: <http://eprint.iacr.org/> (访问日期: 2019年2月28日)。
44. Apecechea, G. I.; Eisenbarth, T.; Sunar, B. S\$A: Shared Cache Attack That 跨核心运行并突破虚拟机沙箱技术——及其在AES中的应用。发表于2015年IEEE安全与隐私研讨会论文集 (SP2015), 美国加利福尼亚州圣何塞, 2015年5月17-21日, 第591-604页。
45. Inci, M.S.; Gulmezoglu, B.; Irazoqui, G.; Eisenbarth, T.; Sunar, B.缓存攻击使云环境下的批量密钥恢复成为可能。收录于《密码硬件与嵌入式系统会议论文集——CHES 2016: 第18届国际会议》, 美国加利福尼亚州圣巴巴拉, 2016年8月17-19日。
46. Daemen, J.; Rijmen, V. 《Rijndael 密码算法设计: 高级加密标准》; 信息安全与密码学; 斯普林格出版社: 德国海德堡/柏林, 2002年。
47. Gulmezoglu, B.; Inci, M.S.; Irazoqui, G.; Eisenbarth, T.; Sunar, B.一种更快且更真实的AES刷洗+重载攻击。收录于《国际构造性侧信道分析与安全设计研讨会论文集——COSADE, 德国柏林, 2015年4月13-14日》; 施普林格出版社: 海德堡/柏林, 2015年; 第9064卷, 第111-126页。
48. 英特尔。英特尔® 64和IA-32 Architectures Optimization Reference Manual (第2.1.1.2节)。2017年。在线获取: <https://software.intel.com/sites/default/files/managed/9e/bc/64-ia-32-architectures-优化手册.pdf> (2019年2月28日访问)。
49. Neve, M.; Tiri, K. 《AES-256 侧 Channel Attacks 复杂性研究——Cache Attacks 方法论与 Quantitative Results》。《密码学 Print Archive》2007年第318期。2007年。在线获取: <http://eprint.iacr.org/> (2019年2月28日访问)。
50. Aciicmez, O.; Schindler, W.; Koc, C.K. 基于缓存的AES远程定时攻击。收录于《密码学专题会议论文集——CT-RSA》, 美国加利福尼亚州旧金山, 2007年2月5-9日; 施普林格出版社: 德国海德堡/柏林, 2007年; 第4377卷, 第271-286页。
51. Biryukov, A.; Khovratovich, D.; Nikolić, I. 针对完整AES-256的区分器及相关密钥攻击研究。收录于《国际密码学年会论文集: 密码学进展》, 美国加利福尼亚州圣巴巴拉, 2009年8月16-20日; 施普林格出版社: 德国海德堡/柏林, 2009年; 第231-249页。
52. Biryukov, A.; Khovratovich, D. 相关——Full AES -192与AES-256的 Key Cryptanalysis。密码学e Print Archive, 报告2009/317。2009年。在线获取: <http://eprint.iacr.org/> (访问日期: 2019年2月28日)。
53. Biryukov, A.; Dunkelman, O.; Keller, N.; Khovratovich, D.; Shamir, A. AES-256变体在最多10轮攻击下的实用复杂度密钥恢复攻击。收录于《密码学进展——欧洲密码学会议论文集》, 2010年5月30日至6月3日, 法国里维埃拉; 施普林格出版社: 德国海德堡/柏林, 2010年; 第6110卷, 第299-319页。
54. Biryukov, A.; Khovratovich, D. AES-256 13轮算法的可行攻击. 加密学电子预印本档案, 报告2010/257. 2010. 在线获取: <http://eprint.iacr.org/> (2019年2月28日访问)。
55. 博格达诺夫 (A. Bogdanov)、霍夫拉托维奇 (D. Khovratovich) 与雷希伯格 (C. Rechberger) 合著的《全AES的双圈密码分析》一文, 收录于《密码学进展——亚洲密码学会议论文集》(2011年12月4-8日, 韩国首尔), 由德国斯普林格出版社 (海德堡/柏林) 出版, 第7073卷, 第344-371页。

56. Kim, C.H. AES的差异性故障分析：减少故障数量的研究。《信息科学》**2012**, *199*, 43–57。 [CrossRef](#)
57. Yan, M.; Sprabery, R.; Gopireddy, B.; Fletcher, C.; Campbell, R.; Torrellas, J. 攻击目标为目录而非缓存：非完全安全环境下的侧信道攻击。2019年IEEE安全与隐私研讨会（SP）（2019），美国加利福尼亚州旧金山，2019年5月20日至22日。



©2019年作者版权所有。授权方 MDPI，瑞士巴塞尔。本文为开放获取文章，根据知识共享署名（CCBY）许可协议（<http://creativecommons.org/licenses/by/4.0/>）条款发布。